Fastify + Temporal

Durable HTTP APIs with Node.js

A companion to the Fastify and Temporal tutorials: put a fast, validated Fastify API in front of durable Temporal workflows — idempotent requests, asynchronous request-reply, long-polling, signals over HTTP, error mapping, health checks, end-to-end tests and failure drills.

Verified with Fastify 5.12 · @temporalio/* 1.24 · Node.js 24 · TypeScript 7

Contents

Part I — Architecture 3

1. Why Combine Fastify and Temporal? . . . 3
2. Architecture . . . 3
3. Designing HTTP for Long-Running Work . . . 4
4. Tested With . . . 5

Part II — Build It 6

5. Project Setup . . . 6
6. The Workflow Side (Reused) . . . 6
7. The Temporal Client Plugin . . . 7
8. Schemas . . . 7
9. Routes: HTTP to Workflow Operations . . . 8
10. App Builder and Error Mapping . . . 11

Part III — Test and Run 14

11. End-to-End Tests . . . 14
12. Run It . . . 16
13. Failure Drills . . . 17

Part IV — Production Notes 19

14. Deployment Topology . . . 19
15. Timeouts, Auth and Observability . . . 19
16. Checklist . . . 19

Appendices 20

A. Endpoint Reference . . . 20
B. Troubleshooting . . . 20

Part I — Architecture

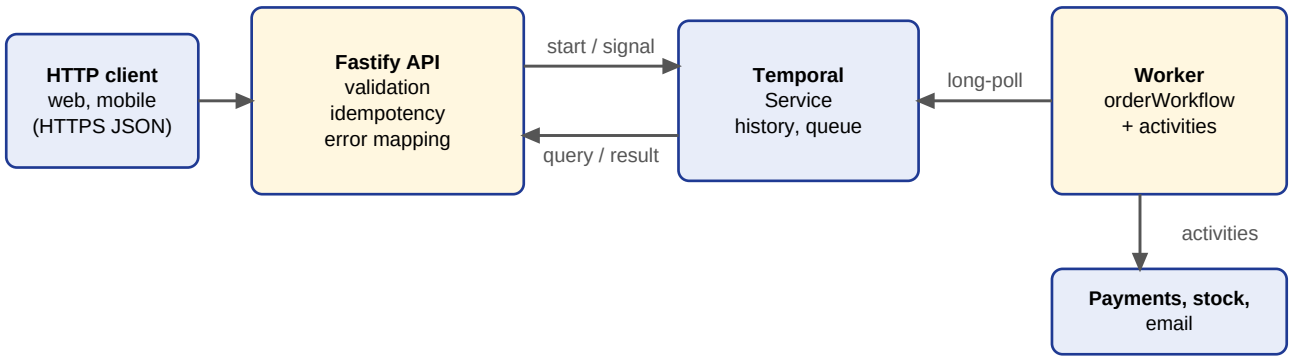
1. Why Combine Fastify and Temporal?

Each tool is excellent at one side of the problem. **Fastify** is the edge: it validates input, authenticates callers, documents the API and answers in milliseconds. **Temporal** is the engine: it runs multi-step business processes that must finish — with retries, timers, human approvals and compensation — even across crashes and deploys.

An HTTP request should not run a 3-day approval process, and a workflow should not parse JSON bodies. Put each concern where it belongs:

Concern	Lives in	Why
Input validation, auth, rate limits	Fastify	Reject bad requests before any durable work starts
API docs (OpenAPI)	Fastify	Generated from the same schemas
Idempotency	Both	Fastify turns the Idempotency-Key into a Workflow ID; Temporal enforces uniqueness
Multi-step process, state	Temporal workflow	Durable, replayable, survives crashes
Calls to external systems, retries	Temporal activities	Declarative retry policies, heartbeats, timeouts
Waiting (approvals, timers)	Temporal workflow	Days-long waits cost nothing
Progress / status for clients	Workflow query, exposed by Fastify	Single source of truth
Mapping outcomes to HTTP codes	Fastify error handler	404 / 422 / 503 in one place

2. Architecture



The API never executes business steps itself. It only talks to the Temporal Service through the client (start, signal, query, result). Workers — a separate process — pull tasks and do the work. You can scale, deploy and restart API and workers independently.

2.1 Life of an order

#	HTTP	What happens
1	`POST /orders` + Idempotency-Key	Schema validation, then `workflow.start()` with Workflow ID `order-{key}`. Returns **202** + Location immediately.
2	—	A worker picks up the task and runs the workflow: reserve stock, charge (with retries), ship, email. Orders over 1000 wait for approval.
3	`GET /orders/{id}`	Describe + workflow query → business status (e.g. AWAITING_APPROVAL) and execution status (RUNNING).
4	`POST /orders/{id}/approve`	Sends a signal. Stored durably even if no worker is running.
5	`GET /orders/{id}/result?waitSeconds=10`	Long-poll: 200 with the result if it completes in time, otherwise 202 with the current status.
6	—	Failure → 422 with the business reason; unknown ID → 404; Temporal down → 503 + Retry-After.

3. Designing HTTP for Long-Running Work

3.1 Asynchronous request-reply

A durable process can take seconds or weeks, so the API must not hold the connection open. The standard pattern: accept the work with **202 Accepted**, return a **Location** to poll, and let clients optionally long-poll for a bounded time (`waitSeconds`, max 30).

3.2 Mapping Temporal outcomes to HTTP

Situation	Temporal signal	HTTP response
Order accepted	`workflow.start()` ok	202 + Location
Same Idempotency-Key again	USE_EXISTING / REJECT_DUPLICATE	202, same Workflow ID, no new run
Status requested	describe + query	200 {status, execution}
Status while no worker runs	query exceeds 5 s deadline	200 with status UNKNOWN
Result not ready	RUNNING or long-poll deadline	202 with current status
Completed	`result()`	200 {result}
Business failure	`WorkflowFailedError`	422 + root-cause message
Unknown order	`WorkflowNotFoundError`	404
Temporal unreachable / too slow	gRPC UNAVAILABLE / DEADLINE_EXCEEDED	503 + Retry-After: 5
Invalid input	— (never reaches Temporal)	400

3.3 Idempotency for free

Clients (and proxies) retry POSTs. Using the `Idempotency-Key` header as the Workflow ID makes Temporal the arbiter: while the order runs, a retry attaches to the existing run (`USE_EXISTING`); after it finished, the server rejects a second run (`REJECT_DUPLICATE`) and the API answers with the same order. No idempotency table, no race conditions.

4. Tested With

The complete project in Parts II–III was type-checked, tested (7 end-to-end tests) and run live against a Temporal dev server, including the failure drills in Chapter 11. It lives in `fastify-temporal-orders/` next to this PDF.

Component	Version	Notes
Node.js	24.15	20+ required by Fastify 5
`fastify`	5.12.5	with sensible, swagger 9.9, swagger-ui 6.1
`@fastify/type-provider-typebox`	6.1.0	TypeBox 1.x
`@temporalio/*`	1.24.0	client, worker, workflow, activity, testing
Temporal CLI / Server	1.9.1 / 1.32.0	dev server
TypeScript / tsx	7.0.2 / 4.23	run with tsx; tests with `node --test`

Part II — Build It

5. Project Setup

```
mkdir fastify-temporal-orders && cd fastify-temporal-orders && npm init -y
npm install fastify fastify-plugin @fastify/sensible @fastify/swagger \
            @fastify/swagger-ui @fastify/type-provider-typebox close-with-grace \
            @temporalio/client @temporalio/worker @temporalio/workflow \
            @temporalio/activity @temporalio/common
npm install -D typescript tsx @types/node pino-pretty @temporalio/testing
```

package.json scripts

```
"scripts": {
  "dev-server": "tsx scripts/dev-server.ts",
  "worker": "tsx src/worker.ts",
  "api": "tsx watch src/server.ts",
  "build": "tsc",
  "test": "node --import tsx --test test/*.test.ts"
}
```

Layout (tsconfig.json: module / moduleResolution "nodenext", as in both guides)

```
fastify-temporal-orders/
  src/
    shared.ts  activities.ts  workflows.ts  worker.ts   # from the Temporal guide
    plugins/temporal.ts      # Temporal client as a Fastify decorator
    schemas/order.ts         # TypeBox schemas (validation, types, docs)
    routes/orders.ts         # HTTP <-> workflow operations
    routes/health.ts         # readiness incl. Temporal reachability
    app.ts                   # buildApp(): plugins, error mapping, routes
    server.ts                # listen + graceful shutdown
  scripts/dev-server.ts      # Temporal dev server without installing the CLI
  test/orders.test.ts        # HTTP -> workflow end-to-end tests
```

6. The Workflow Side (Reused)

The workflow, activities and worker are the **unchanged** files from Part VI of the Temporal tutorial (explained there line by line). What the API relies on:

- Task queue `orders`; workflow `orderWorkflow(input)`.
- Signals `approveOrder(approver)` and `cancelOrder()`; query `getStatus`.
- Orders above 1000 wait up to 2 minutes for approval, then cancel themselves.
- Out-of-stock throws a non-retryable `OutOfStock` failure; the payment activity fails twice before succeeding (simulated).

```
// src/shared.ts
export const TASK_QUEUE = 'orders';

export interface OrderInput {
  orderId: string;
  customerEmail: string;
  sku: string;
  quantity: number;
  amount: number;          // in dollars
}

export type OrderStatus =
  | 'RECEIVED' | 'AWAITING_APPROVAL' | 'RESERVED' | 'CHARGED'
  | 'SHIPPED' | 'CANCELLED' | 'FAILED';
```

7. The Temporal Client Plugin

One Temporal client per process, created at boot and shared through a decorator. The plugin accepts a ready-made client so tests can inject the time-skipping server.

```ts
// src/plugins/temporal.ts
import fp from 'fastify-plugin';
import { Connection, Client } from '@temporalio/client';

declare module 'fastify' {
  interface FastifyInstance { temporal: Client }  1
}

export interface TemporalPluginOptions {
  client?: Client;          // pass a ready client (tests, shared connections)  2
  address?: string;
  namespace?: string;
}

export default fp<TemporalPluginOptions>(async (app, opts) => {
  if (opts.client) {
    app.decorate('temporal', opts.client);   // caller owns the connection  3
    return;
  }
  const address = opts.address ?? 'localhost:7233';
  const connection = await Connection.connect({ address });  4
  const namespace = opts.namespace ?? 'default';
  app.decorate('temporal', new Client({ connection, namespace }));
  app.addHook('onClose', async () => { await connection.close(); });  5
  app.log.info({ address, namespace }, 'connected to Temporal');
}, { name: 'temporal' });  6
```

1	Every route and hook can use `app.temporal` with full typing.
2	Dependency injection: tests pass the time-skipping server's client, production passes an address.
3	An injected client is owned by the caller, so the plugin must not close it.
4	Connect once at boot. The client reconnects on its own after Temporal restarts (verified: health went 503 → 200 without restarting the API).
5	`app.close()` stops HTTP first, then closes the gRPC connection.
6	Wrapped in `fp()` so the decorator is visible to all route plugins.

8. Schemas

```ts
// src/schemas/order.ts
import { Type, Static } from '@fastify/type-provider-typebox';

export const CreateOrder = Type.Object({
  customerEmail: Type.String({ format: 'email' }),
  sku: Type.String({ minLength: 1 }),
  quantity: Type.Integer({ minimum: 1, maximum: 100 }),
  amount: Type.Number({ exclusiveMinimum: 0 }),
}, { additionalProperties: false });  1
export type CreateOrder = Static<typeof CreateOrder>;

export const IdempotencyHeaders = Type.Object({
  'idempotency-key': Type.Optional(Type.String({ pattern: '^[A-Za-z0-9_-]{8,64}$' })),  2
});
```

1	Validated at the edge: bad input never reaches Temporal or creates a workflow.
2	The key becomes part of the Workflow ID, so it is restricted to a safe charset.

```
export const OrderParams = Type.Object({ id: Type.String({ pattern: '^order-' }) });  3

export const Accepted = Type.Object({
  workflowId: Type.String(), statusUrl: Type.String(),
});

export const OrderView = Type.Object({
  workflowId: Type.String(),
  status: Type.String(),        // business status from the workflow query  4
  execution: Type.String(),     // RUNNING | COMPLETED | FAILED | ...
});

export const Approve = Type.Object({ approver: Type.String({ minLength: 1 }) },
                                   { additionalProperties: false });
```

3 Only IDs this API created are accepted: `/orders/nope` is a 400 without a round-trip to Temporal.

4 Two statuses: the **business** state (from the workflow query) and the **execution** state (from Temporal).

```
export const ResultQuery = Type.Object({
  waitSeconds: Type.Integer({ minimum: 0, maximum: 30, default: 0 }),  5
});

export const ResultReply = Type.Object({
  workflowId: Type.String(), result: Type.String(),
});

export const ErrorReply = Type.Object({
  statusCode: Type.Integer(), error: Type.String(), message: Type.String(),
});
```

5 Optional long-poll, capped at 30 s so HTTP connections are never held indefinitely.

9. Routes: HTTP to Workflow Operations

```
// src/routes/orders.ts
import type { FastifyPluginAsyncTypebox } from '@fastify/type-provider-typebox';
import {
  WorkflowExecutionAlreadyStartedError, isGrpcDeadlineError,
} from '@temporalio/client';
import { randomUUID } from 'node:crypto';
import { orderWorkflow, approveOrder, cancelOrder, getStatus } from '../workflows';
import { TASK_QUEUE } from '../shared';
import {
  CreateOrder, IdempotencyHeaders, OrderParams, Accepted, OrderView, Approve,
  ResultQuery, ResultReply, ErrorReply,
} from '../schemas/order';

const orders: FastifyPluginAsyncTypebox = async (app) => {
  const common = { tags: ['orders'] };
```

```
// Reads the workflow's own view of the order; never hangs without a worker
async function view(workflowId: string) {  1
  const handle = app.temporal.workflow.getHandle(workflowId);
  const desc = await handle.describe();
  const status = await app.temporal.connection
    .withDeadline(Date.now() + 5_000, () => handle.query(getStatus))  2
    .catch(() => 'UNKNOWN (no worker available)');
  return { workflowId, status, execution: desc.status.name };
}

app.post('/', {
  schema: { ...common, body: CreateOrder, headers: IdempotencyHeaders,
            response: { 202: Accepted, 400: ErrorReply } },
}, async (request, reply) => {
  const key = request.headers['idempotency-key'] ?? randomUUID();  3
  const workflowId = `order-${key}`;
  try {
    // Bounded: fail fast (503) instead of letting the client retry for ~20 s
    await app.temporal.connection.withDeadline(Date.now() + 5_000, () =>  4
      app.temporal.workflow.start(orderWorkflow, {
        taskQueue: TASK_QUEUE,
        workflowId,
        args: [{ orderId: key, ...request.body }],
        workflowIdConflictPolicy: 'USE_EXISTING',   // same key while running  5
        workflowIdReusePolicy: 'REJECT_DUPLICATE',  // same key after it finished  6
      }));
  } catch (err) {
    if (!(err instanceof WorkflowExecutionAlreadyStartedError)) throw err;  7
    request.log.info({ workflowId }, 'idempotent replay of a finished order');
  }
  const statusUrl = `/orders/${workflowId}`;
  reply.code(202).header('location', statusUrl);  8
  return { workflowId, statusUrl };
});
```

1	One place that builds the status view; reused by GET and by 202 responses.
2	Queries need a live worker. Bounded to 5 s, then degrade to `UNKNOWN` instead of hanging the request.
3	Client-supplied key (or a fresh UUID) becomes the Workflow ID: **Temporal enforces the idempotency**, no extra table needed.
4	Without this bound the client retries for ~20 s when Temporal is down (measured). With it the API answers 503 in ~5 s.
5	Same key while the order is running: attach to the existing run.
6	Same key after it finished: the server refuses a second run...
7	...which we treat as success. Tests assert the **run ID** is unchanged.
8	Asynchronous request-reply: 202 Accepted + Location; the client polls the status URL.

```
  app.get('/:id', {
    schema: { ...common, params: OrderParams,
              response: { 200: OrderView, 404: ErrorReply } },
  }, async (request) => view(request.params.id));

  app.post('/:id/approve', {
    schema: { ...common, params: OrderParams, body: Approve,
              response: { 202: Accepted, 404: ErrorReply } },
  }, async (request, reply) => {
    const { id } = request.params;
    const handle = app.temporal.workflow.getHandle(id);
    await handle.signal(approveOrder, request.body.approver);  9
    reply.code(202);
    return { workflowId: id, statusUrl: `/orders/${id}` };
  });
```

9 Signals map to POST sub-resources. Stored by the server even if no worker is up; a missing workflow throws `WorkflowNotFoundError` → 404.

```
  app.post('/:id/cancel', {
    schema: { ...common, params: OrderParams,
              response: { 202: Accepted, 404: ErrorReply } },
  }, async (request, reply) => {
    const { id } = request.params;
    await app.temporal.workflow.getHandle(id).signal(cancelOrder);
    reply.code(202);
    return { workflowId: id, statusUrl: `/orders/${id}` };
  });

  app.get('/:id/result', {
    schema: { ...common, params: OrderParams, querystring: ResultQuery,
              response: { 200: ResultReply, 202: OrderView, 404: ErrorReply,
                          422: ErrorReply } },
  }, async (request, reply) => {
    const { id } = request.params;
    const handle = app.temporal.workflow.getHandle(id);
    const running = (await handle.describe()).status.name === 'RUNNING';
    if (running && request.query.waitSeconds === 0) {  10
      return reply.code(202).send(await view(id));
    }
    try {
      const result = running
        ? await app.temporal.connection.withDeadline(
            Date.now() + request.query.waitSeconds * 1000, () => handle.result())  11
        : await handle.result();
      return { workflowId: id, result };
    } catch (err) {
      if (!isGrpcDeadlineError(err)) throw err;            // walks the cause chain  12
      return reply.code(202).send(await view(id));     // still running: poll again
    }
  });
};

export default orders;
```

10 Default is a cheap non-blocking check.

11 Long-poll for up to `waitSeconds`: `result()` returns as soon as the workflow completes.

12 The deadline error arrives wrapped in a `ServiceError`; `isGrpcDeadlineError` checks the cause chain (checking `err.code` directly misses it).

10. App Builder and Error Mapping

```ts
// src/app.ts
import Fastify, { FastifyServerOptions, FastifyError } from 'fastify';
import { STATUS_CODES } from 'node:http';
import sensible from '@fastify/sensible';
import swagger from '@fastify/swagger';
import swaggerUi from '@fastify/swagger-ui';
import type { TypeBoxTypeProvider } from '@fastify/type-provider-typebox';
import {
  WorkflowNotFoundError, WorkflowFailedError, isGrpcServiceError, isGrpcDeadlineError,
} from '@temporalio/client';
import temporal, { TemporalPluginOptions } from './plugins/temporal';
import health from './routes/health';
import orders from './routes/orders';

const GRPC_UNAVAILABLE = 14;  // 1
```

1 gRPC status UNAVAILABLE: the Temporal Service cannot be reached.

```ts
// Temporal Service unreachable or too slow: the caller should retry later
function temporalUnavailable(err: unknown): boolean {  // 2
  if (isGrpcDeadlineError(err)) return true;
  for (let e: unknown = err; e instanceof Error; e = e.cause) {
    if (isGrpcServiceError(e) && e.code === GRPC_UNAVAILABLE) return true;
  }
  return false;
}

export interface AppOptions extends FastifyServerOptions {
  temporal?: TemporalPluginOptions;  // 3
}

export async function buildApp(
  { temporal: temporalOpts = {}, ...opts }: AppOptions = {},
) {
  const app = Fastify(opts).withTypeProvider<TypeBoxTypeProvider>();
```

2 Both "unreachable" and "too slow" mean the same to an HTTP client: retry later.

3 App options carry Temporal settings, so tests and production build the same app.

```
  await app.register(sensible);
  await app.register(swagger, {
    openapi: { info: { title: 'Orders API (Fastify + Temporal)', version: '1.0.0' } },
  });
  await app.register(swaggerUi, { routePrefix: '/docs' });
  await app.register(temporal, temporalOpts);  4

  // Translate Temporal outcomes into HTTP semantics in one place
  app.setErrorHandler((error: FastifyError, request, reply) => {
    if (error instanceof WorkflowNotFoundError) {  5
      return reply.code(404).send({ statusCode: 404, error: 'Not Found',
                                    message: 'Order not found' });
    }
    if (error instanceof WorkflowFailedError) {  6
      let cause: Error | undefined = error.cause;      // unwrap ActivityFailure etc.
      while (cause?.cause instanceof Error) cause = cause.cause;
      return reply.code(422).send({ statusCode: 422, error: 'Unprocessable Entity',
                                    message: cause?.message ?? 'Order failed' });
    }
    if (temporalUnavailable(error)) {
      request.log.warn({ err: error }, 'Temporal unavailable');
      return reply.code(503).header('retry-after', '5').send({  7
        statusCode: 503, error: 'Service Unavailable',
        message: 'Temporarily unavailable, retry with the same Idempotency-Key',
      });
    }
    const status = error.statusCode ?? 500;
    if (status >= 500) request.log.error({ err: error }, 'unhandled error');
    reply.code(status).send({
      statusCode: status,
      error: STATUS_CODES[status] ?? 'Error',
      message: status >= 500 ? 'Something went wrong' : error.message,  8
    });
  });
```

4	Registered before the routes that use `app.temporal`.
5	Unknown Workflow ID → 404.
6	A failed workflow is a business outcome → 422 with the root cause (e.g. `Out of stock: LAPTOP`).
7	Temporal down → 503 + Retry-After. Safe to retry because POST is idempotent.
8	Everything else: standard Fastify behaviour, 5xx details hidden.

```
  await app.register(health);
  await app.register(orders, { prefix: '/orders' });
  return app;
}
```

10.1 Health check and server

```ts
// src/routes/health.ts
import type { FastifyPluginAsync } from 'fastify';

const health: FastifyPluginAsync = async (app) => {
  app.get('/health', { schema: { tags: ['system'] } }, async (request, reply) => {
    try {
      await app.temporal.connection.withDeadline(Date.now() + 2_000,
        () => app.temporal.workflowService.getSystemInfo({})); // 1
      return { status: 'ok', temporal: 'reachable' };
    } catch (err) {
      request.log.warn({ err }, 'Temporal health check failed');
      return reply.code(503).send({ status: 'degraded', temporal: 'unreachable' }); // 2
    }
  });
};

export default health;
```

1. A cheap call that proves the Temporal Service is reachable.
2. Readiness probes take the instance out of rotation while Temporal is unreachable.

```ts
// src/server.ts
import closeWithGrace from 'close-with-grace';
import { buildApp } from './app';

async function main() {
  const app = await buildApp({
    logger: {
      level: process.env.LOG_LEVEL ?? 'info',
      transport: process.env.NODE_ENV === 'production'
        ? undefined : { target: 'pino-pretty' },
    },
    temporal: {
      address: process.env.TEMPORAL_ADDRESS ?? 'localhost:7233', // 1
      namespace: process.env.TEMPORAL_NAMESPACE ?? 'default',
    },
  });
```

1. Same env variables as the worker, so both processes point at the same Temporal Service.

```ts
  closeWithGrace({ delay: 10_000 }, async ({ signal, err }) => {
    if (err) app.log.error({ err }, 'shutting down after error');
    else app.log.info({ signal }, 'shutting down');
    await app.close();              // stops HTTP, then closes the Temporal connection // 2
  });

  await app.listen({ port: Number(process.env.PORT ?? 3000), host: '0.0.0.0' }); // 3
}

main().catch((err) => { console.error(err); process.exit(1); });
```

2. Graceful shutdown order: drain HTTP requests, then run `onClose` hooks (Temporal connection).
3. The worker is a **separate process** (`npm run worker`): scale and deploy API and workers independently.

Part III — Test and Run

11. End-to-End Tests

The tests drive the real Fastify app with `inject()`, which calls the real Temporal client, which runs the real workflow on a real worker — only the activities are mocked. Timers are skipped, so the suite runs in seconds.

```ts
// test/orders.test.ts
// End-to-end: HTTP (inject) -> Fastify -> Temporal client -> workflow
//             -> mocked activities, on a time-skipping test server
import { test, before, after } from 'node:test';
import assert from 'node:assert/strict';
import { TestWorkflowEnvironment } from '@temporalio/testing';
import { Worker } from '@temporalio/worker';
import { ApplicationFailure } from '@temporalio/common';
import { buildApp } from '../src/app';
import { TASK_QUEUE } from '../src/shared';

let env: TestWorkflowEnvironment;
let app: Awaited<ReturnType<typeof buildApp>>;
let worker: Worker;
let workerRun: Promise<void>;
```

```ts
const activities = {
  reserveInventory: async (_id: string, sku: string) => {
    if (sku === 'NONE') {
      throw ApplicationFailure.nonRetryable('Out of stock: NONE', 'OutOfStock');
    }
  },
  releaseInventory: async () => {},
  chargePayment: async (id: string) => `txn-${id}`,
  refundPayment: async () => {},
  shipOrder: async (id: string) => `TRACK-${id}`,
  sendEmail: async () => {},
};

before(async () => {
  env = await TestWorkflowEnvironment.createTimeSkipping();  1
  worker = await Worker.create({
    connection: env.nativeConnection, taskQueue: TASK_QUEUE,
    workflowsPath: require.resolve('../src/workflows'), activities,
  });
  workerRun = worker.run();  2
  app = await buildApp({ temporal: { client: env.client } });   // inject test client  3
});
```

1	In-process Temporal test server with time skipping (2-minute timers take ms).
2	A real worker runs the real workflow code, with mocked activities.
3	The Fastify app uses the test server's client: HTTP → workflow end to end, no ports opened.

```
after(async () => {
  await app.close();
  worker.shutdown();  4
  await workerRun;
  await env.teardown();
});

const order = { customerEmail: 'ann@example.com', sku: 'BOOK', quantity: 1,
                amount: 25 };
const post = (payload: object, key?: string) => app.inject({
  method: 'POST', url: '/orders', payload,
  headers: key ? { 'idempotency-key': key } : {},
});

test('rejects an invalid order with 400', async () => {
  const res = await post({ ...order, quantity: 0 });
  assert.equal(res.statusCode, 400);
});
```

4 Stop the worker, wait for it, then tear down the server.

```
test('small order: 202 + Location, then result', async () => {
  const res = await post(order);
  assert.equal(res.statusCode, 202);
  const { workflowId, statusUrl } = res.json();
  assert.equal(res.headers.location, statusUrl);

  const result = await app.inject(`${statusUrl}/result?waitSeconds=10`);
  assert.equal(result.statusCode, 200);
  assert.match(result.json().result, /^TRACK-/);
  assert.equal((await app.inject(statusUrl)).json().status, 'SHIPPED');
  assert.ok(workflowId.startsWith('order-'));
});

test('Idempotency-Key returns the same order, even after it finished', async () => {
  const key = 'checkout-12345';
  const runId = async (id: string) =>
    (await env.client.workflow.getHandle(id).describe()).runId;
```

```
  const a = (await post(order, key)).json();
  const firstRun = await runId(a.workflowId);  5
  const b = (await post(order, key)).json();                    // while running
  assert.equal(b.workflowId, a.workflowId);
  assert.equal(await runId(b.workflowId), firstRun);

  await app.inject(`${a.statusUrl}/result?waitSeconds=10`);     // let it finish
  const c = await post(order, key);                          // after it finished
  assert.equal(c.statusCode, 202);
  assert.equal(await runId(c.json().workflowId), firstRun);     // no second run
});

test('large order waits for approval given over HTTP', async () => {
  const { statusUrl } = (await post({ ...order, amount: 5000 })).json();
  assert.equal((await app.inject(statusUrl)).json().status, 'AWAITING_APPROVAL');
```

5 Idempotency is proven by comparing Temporal **run IDs**, not just the returned Workflow ID.

```
  const pending = await app.inject(`${statusUrl}/result`);          // waitSeconds=0  6
  assert.equal(pending.statusCode, 202);

  const approve = await app.inject({ method: 'POST', url: `${statusUrl}/approve`,
                                     payload: { approver: 'manager-jo' } });
  assert.equal(approve.statusCode, 202);
  const done = await app.inject(`${statusUrl}/result?waitSeconds=10`);
  assert.equal(done.statusCode, 200);
});

test('failed order maps to 422 with the business reason', async () => {
  const { statusUrl } = (await post({ ...order, sku: 'NONE' })).json();
  const res = await app.inject(`${statusUrl}/result?waitSeconds=10`);
  assert.equal(res.statusCode, 422);
  assert.equal(res.json().message, 'Out of stock: NONE');  7
});
```

6	Non-blocking check returns 202 while the order waits for approval.
7	The 422 carries the activity's business reason, unwrapped from the failure chain.

```
test('unknown order maps to 404', async () => {
  const res = await app.inject('/orders/order-does-not-exist');
  assert.equal(res.statusCode, 404);
  const sig = await app.inject({ method: 'POST', url: '/orders/order-nope/cancel' });
  assert.equal(sig.statusCode, 404);
});

test('health reports Temporal reachable', async () => {
  const res = await app.inject('/health');
  assert.equal(res.statusCode, 200);
  assert.equal(res.json().temporal, 'reachable');
});
```

Output

```
npm test
# ok  rejects an invalid order with 400
# ok  small order: 202 + Location, then result
# ok  Idempotency-Key returns the same order, even after it finished
# ok  large order waits for approval given over HTTP
# ok  failed order maps to 422 with the business reason
# ok  unknown order maps to 404
# ok  health reports Temporal reachable
# tests 7, pass 7, fail 0
```

12. Run It

```
npm run dev-server     # terminal 1 (or: temporal server start-dev), UI :8233
npm run worker         # terminal 2
npm run api            # terminal 3, http://localhost:3000, docs at /docs
```

Real responses from the verification run (Windows: use `curl.exe`):

```
# 1. Create with an idempotency key -> 202 + Location
curl -i -X POST localhost:3000/orders -H "Content-Type: application/json" \
     -H "Idempotency-Key: demo-order-0001" \
     -d '{"customerEmail":"ann@example.com","sku":"BOOK","quantity":1,"amount":25}'
# HTTP/1.1 202 Accepted
# location: /orders/order-demo-order-0001
# {"workflowId":"order-demo-order-0001","statusUrl":"/orders/order-demo-order-0001"}

# 2. Non-blocking result check while it runs -> 202
curl localhost:3000/orders/order-demo-order-0001/result
# {"workflowId":"order-demo-order-0001","status":"RECEIVED","execution":"RUNNING"}

# 3. Long-poll up to 10 s (payment retries twice, then succeeds) -> 200
curl "localhost:3000/orders/order-demo-order-0001/result?waitSeconds=10"
# {"workflowId":"order-demo-order-0001","result":"TRACK-DEMO-ORDER-0001"}

# 4. Same key again -> same order, no second run
curl -X POST localhost:3000/orders -H "Idempotency-Key: demo-order-0001" ...
# {"workflowId":"order-demo-order-0001","statusUrl":"/orders/order-demo-order-0001"}
```

```
# 5. Large order: long-poll 3 s times out -> 202 with status (took 3 s)
curl "localhost:3000/orders/order-demo-large-0002/result?waitSeconds=3"
# {"workflowId":"order-demo-large-0002","status":"AWAITING_APPROVAL",
#  "execution":"RUNNING"}

# 6. Approve, then wait -> 200
curl -X POST localhost:3000/orders/order-demo-large-0001/approve \
     -H "Content-Type: application/json" -d '{"approver":"manager-jo"}'
# {"workflowId":"order-demo-large-0001","statusUrl":"/orders/order-demo-large-0001"}
curl "localhost:3000/orders/order-demo-large-0001/result?waitSeconds=20"
# {"workflowId":"order-demo-large-0001","result":"TRACK-DEMO-LARGE-0001"}

# 7. Out of stock -> 422 with the business reason
# {"statusCode":422,"error":"Unprocessable Entity","message":"Out of stock: LAPTOP"}

# 8. Validation and unknown IDs
# 400 {"message":"body/customerEmail must match format \"email\"", ...}
# 404 {"message":"Order not found", ...}
# 400 {"message":"params/id must match pattern \"^order-\"", ...}
```

13. Failure Drills

Durability is the point of the combination, so it was tested by breaking things while the API kept running:

Watch out: Two bugs were found only by running these drills: the long-poll timeout surfaced as 500 because the deadline error is wrapped in a `ServiceError` (fixed with `isGrpcDeadlineError`), and order creation hung for 20 s when Temporal was down (fixed with `withDeadline` + 503).

Drill	Observed behaviour
Kill the worker, then `GET /orders/{id}`	200 after 5 s with `status: "UNKNOWN (no worker available)"` — no hanging request
Worker still down: `POST /approve`	202; the signal is stored by Temporal
Restart the worker, long-poll the result	200 `TRACK-DEMO-LARGE-0003`: the buffered approval was applied
Stop the Temporal Service: `GET /health`	503 `{"status":"degraded"}` within 3 s
Temporal down: `POST /orders`	Before the fix: **500 after 20 s** (client retries). After: **503 + Retry-After: 5 in about 5 s**
Restart Temporal, same API process: `GET /health`	200 — the client reconnected on its own
Cancel an order waiting for approval	202, then result `CANCELLED`

Part IV — Production Notes

14. Deployment Topology

- **One image, two commands**: `node dist/server.js` (API) and `node dist/worker.js` (worker). Deploy them as separate services.
- **Scale independently**: API replicas by request rate; workers by Temporal *schedule-to-start latency* (tasks waiting for a worker).
- Use a **glibc** base image (e.g. `node:22-bookworm-slim`) — the worker's native core does not run on Alpine.
- Pre-bundle workflow code for the worker (Temporal guide, Chapter 20.1).
- Same `TEMPORAL_ADDRESS` / `TEMPORAL_NAMESPACE` (and Cloud credentials) for API and worker.

Tip: Running the worker inside the API process is possible for small apps and local development, but it couples crashes, CPU usage and scaling of both. Keep them separate in production.

15. Timeouts, Auth and Observability

- **Align timeouts**: Fastify `requestTimeout` and your load balancer's idle timeout must exceed the maximum long-poll (30 s) plus the 5 s query bound.
- **Authentication**: add `@fastify/jwt` exactly as in the Fastify tutorial (Chapter 16) and take the approver from `request.user` instead of the request body, so the audit trail in the workflow history is trustworthy.
- **Authorization**: store the owner in the workflow input and check it before signalling or exposing status (404 for foreign orders, as in the Tasks API).
- **Correlation**: log the Workflow ID with every request (the POST handler already does for idempotent replays); the Temporal UI then shows the full history for that ID.
- **Metrics**: watch API p95 latency, 503 rate and Temporal schedule-to-start latency together.

16. Checklist

Item	Where
Validate at the edge; reject before starting workflows	TypeBox schemas
Idempotency-Key → Workflow ID with USE_EXISTING + REJECT_DUPLICATE	`POST /orders`
202 + Location; bounded long-poll	`GET /orders/{id}/result`
Bound every Temporal call made on a request path	`connection.withDeadline`
404 / 422 / 503 mapping in one error handler	`app.ts`
Health endpoint checks Temporal reachability	`GET /health`
Graceful shutdown: HTTP first, then Temporal connection	`close-with-grace`
API and worker deployed and scaled separately	Deployment
End-to-end tests with time skipping	`test/orders.test.ts`

Appendices

A. Endpoint Reference

Method & path	Body / query	Responses
`POST /orders`	CreateOrder; header `Idempotency-Key` (optional)	202 Accepted, 400, 503
`GET /orders/{id}`	—	200 OrderView, 404, 503
`POST /orders/{id}/approve`	`{"approver": string}`	202 Accepted, 404, 503
`POST /orders/{id}/cancel`	—	202 Accepted, 404, 503
`GET /orders/{id}/result`	`waitSeconds` 0–30 (default 0)	200 ResultReply, 202 OrderView, 404, 422, 503
`GET /health`	—	200 reachable, 503 degraded
`GET /docs`	—	Swagger UI (JSON at /docs/json)

B. Troubleshooting

Symptom	Cause / fix
Long-poll returns 500 instead of 202	Deadline error is wrapped in `ServiceError`. Use `isGrpcDeadlineError(err)`, not `err.code`.
POST hangs ~20 s when Temporal is down	Client retries internally. Wrap `start()` in `withDeadline` and map to 503.
GET status hangs	Query with no worker running. Bound it with `withDeadline` and return UNKNOWN.
Idempotent POST starts a second run after completion	Default reuse policy allows duplicates. Set `workflowIdReusePolicy: 'REJECT_DUPLICATE'`.
`WorkflowExecutionAlreadyStartedError` as 500	Catch it in the POST handler and return the existing order.
Tests hang on teardown	Call `worker.shutdown()` and await the `run()` promise before `env.teardown()`.
Approval timeout fires instantly in tests	Time skipping: waiting on `result()` fast-forwards timers. Signal before waiting.

Further reading

- Fastify with Node.js — The Complete Practical Tutorial (companion PDF)
- Temporal with Node.js — The Complete Practical Tutorial (companion PDF)
- docs.temporal.io/develop/typescript · fastify.dev/docs/latest